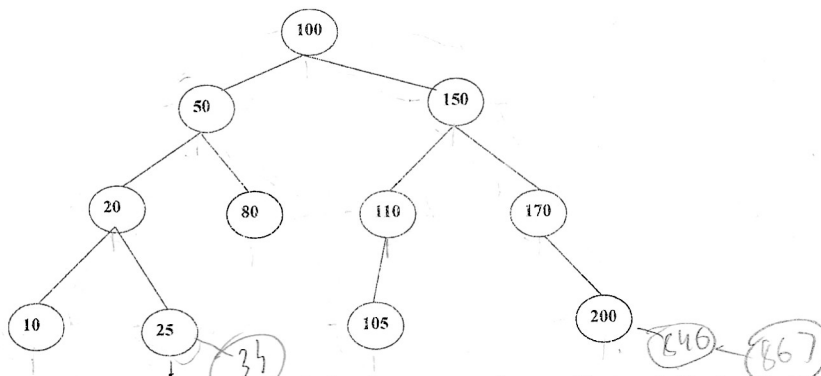


Aluno: Flávio Guilherme Santana Corrêa
UFMA, CCET, Ciência da Computação, Estrutura de Dados I

Terceira Prova

1 Esta questão tem três partes que devem ser feitas em sequência. Cada parte usa como ponto de partida a árvore que resultou da parte anterior. Você nunca deve voltar à árvore original depois da Parte A.



Parte A. Percursos (use a Árvore Original, mostrada na figura) Cada visitação imprime o valor de todos os nós da árvore, em uma ordem diferente. Escreva o resultado de cada visitação abaixo, usando a árvore original da figura:

- a) Pré-ordem b) Pós-ordem c) Ordem simétrica

root, l, r *l, r, root* *l, root, r*
Parte B. Inserção (ponto de partida: Árvore Original)

Comece novamente da Árvore Original da figura. Insira os três valores abaixo, um de cada vez, nesta ordem, desenhando como fica a árvore após cada inserção:

- d) Insira 846 e) Insira 867 f) Insira 34

Parte C. Remoção (ponto de partida: Árvore 2, resultado da Parte B)

Importante: nesta parte, não use mais a Árvore Original. Use a **Árvore gerada na letra f** que é o resultado que você desenhou ao final da Parte B, já com os valores 846, 867 e 34 inseridos. A partir desta Árvore, remova os três valores abaixo, um de cada vez, nesta ordem:

- g) Remova ~~150~~ h) Remova ~~80~~ i) Remova ~~50~~

Desenhe a árvore resultante ao final de cada uma dessas remoções.

2: Diferença entre maior e menor valor. Você deve escrever um algoritmo que recebe uma árvore binária de pesquisa e calcula a diferença entre o maior valor e o menor valor armazenados nela. Regras obrigatórias:

- Use exatamente o protótipo abaixo
- Você deve usar `getvalue` sempre que precisar ler o valor de um nó.

```
int abpCalcDifMaiorParaMenor(TNode *t, int (*getvalue)(void *));
```

3: Contagem de folhas. Você deve escrever um algoritmo que recebe a raiz de uma árvore binária e conta quantas folhas ela possui.

```
int abpNumFolhas(TNode *t);
```

4: Remoção de folha. Você deve escrever um algoritmo que recebe a raiz de uma árvore binária de pesquisa e uma chave, e remove o nó identificado por essa chave, mas apenas se esse nó for uma folha.

Regras obrigatórias: Se o nó encontrado não for uma folha, a função não deve remover nada.

```
int abpRemoveFolha(TNode *t, void *key, int (*cmp)(void *, void *));
```

TNode *